Product Growth Under AI Agents

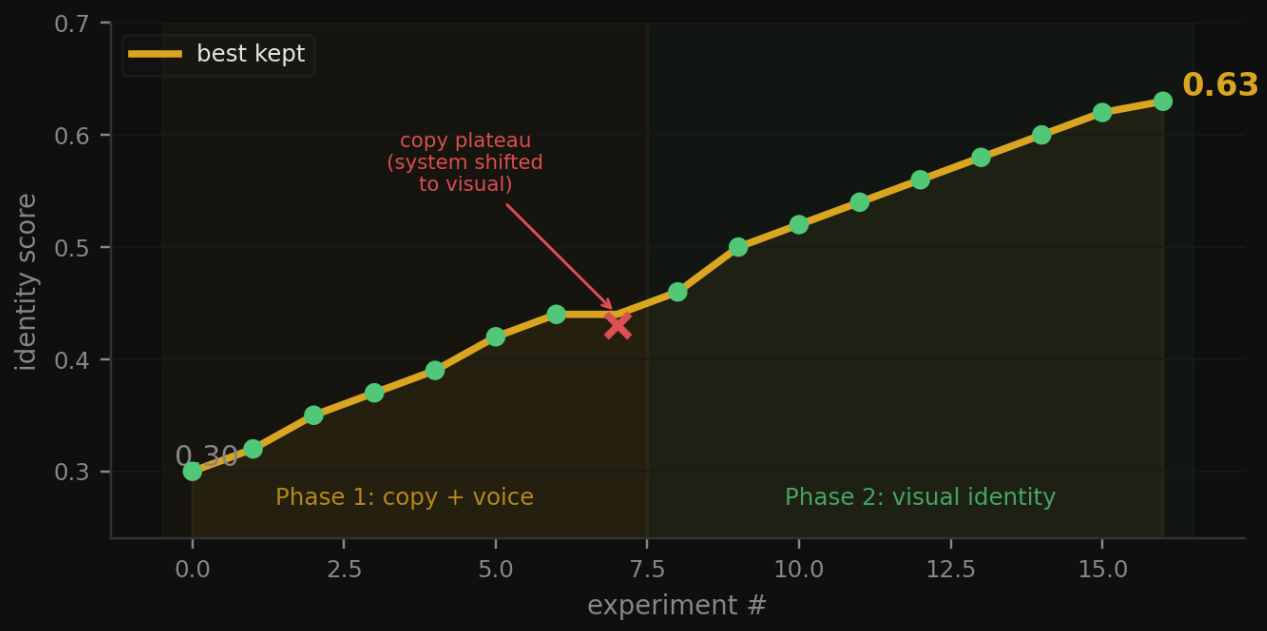
Real trajectories from products managed by rhino-os.
65 experiments. 4 eval cycles. 5 agents. 2 projects.

Identity score	**0.30 → 0.63 (+110%)**
Return pull	**0.20 → 0.63 (+215%)**
Agent uptime	**2/5 → 5/5 (self-healed)**
Architecture	**13 agents → 5 (-62%)**
Experiments run	**65 (3 discarded)**

rhino-os — github.com/rhinehart514/rhino-os

Identity: 0.30 → 0.63 in 17 experiments

The system detected that copy-only changes plateaued at 0.44 and autonomously shifted to visual identity changes, which pushed past the ceiling. One experiment was discarded — a gold ping animation that improved the score but violated product constraints. That judgment call wasn't programmed. It emerged from the rubric.



Scores are self-assessed via rubric. The trend is meaningful. Absolute values should be read with healthy skepticism.

Ceiling dimensions across 4 eval cycles

Return pull showed the biggest improvement (+215%) after a dedicated sprint targeting the day-3 return experience. Identity resolved after the experiment sprint above. Escape velocity remains stubborn — it requires network effects that don't exist yet at this user count. The system correctly identifies this as a stage constraint, not a fixable bug.

3 agents died. The system fixed itself.

On 03-07, a nesting bug (CLAUDECODE env var blocking spawned processes) silently killed 3 of 5 agents. Nobody noticed. On 03-09, the meta agent ran its grading cycle, detected the crashes, diagnosed the root cause, patched the spawn command across all agent prompts, and brought everything back online.

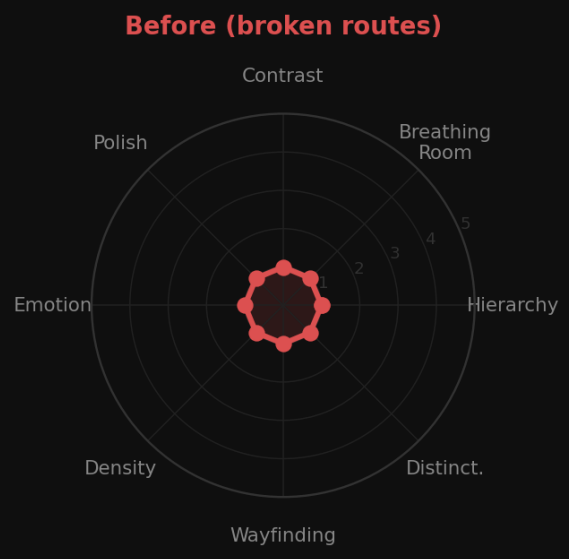
The fix was one line: `CLAUDECODE= claude ...` to clear the environment variable before spawning. Meta found it by reading error logs and tracing the execution path. No human intervention.

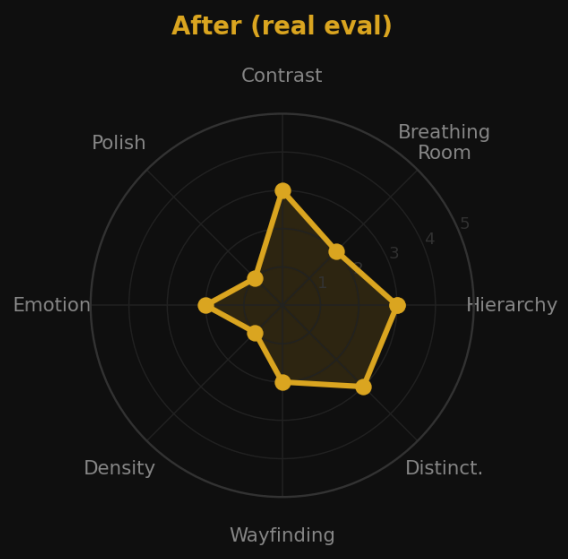
Structural scoring: catching regressions in real time

score.sh runs on every commit. The build dimension dips (red triangles) are real regressions — TypeScript errors, stale builds, broken compilation. Each dip was caught immediately and fixed before it compounded. Structure and hygiene track long-term codebase health. This is the cheap "training loss" — fast, deterministic, every commit.

Visual taste: the expensive eval

The left radar is the first taste eval — routes were broken, everything scored 1/5. Not a design failure — a deployment failure. The system caught it. The right radar is the real eval after routes were fixed. Still mostly 2s and 3s (honest for an early product). The gap between these two tells you what "measuring by looking" catches that grep-based scoring misses.

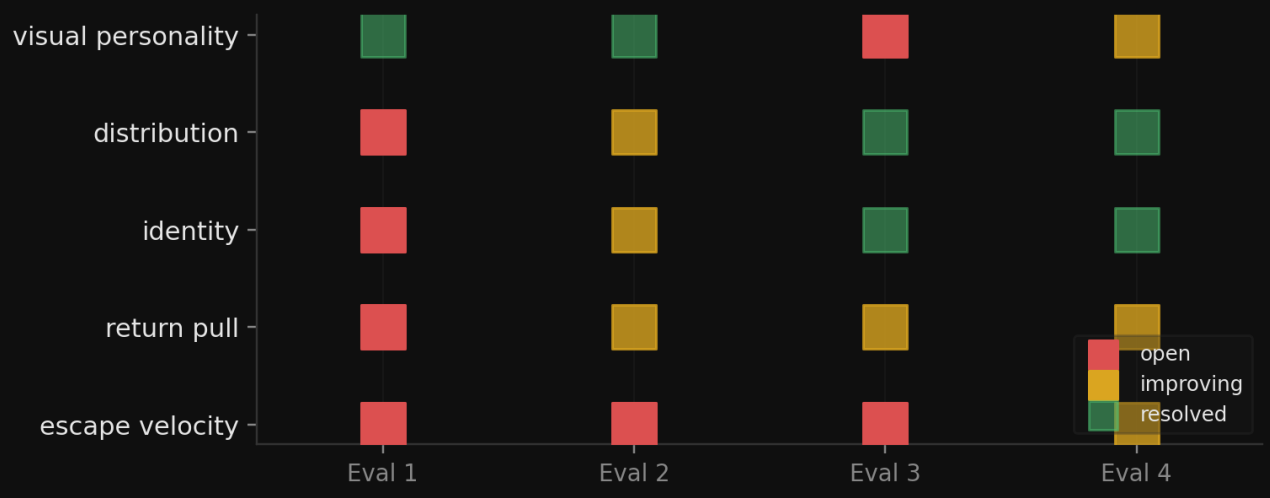




Taste scores are Claude vision reading Playwright screenshots. Calibrated against Notion/Discord/Linear. Most early products score 2-3. A 4+ is genuinely rare.

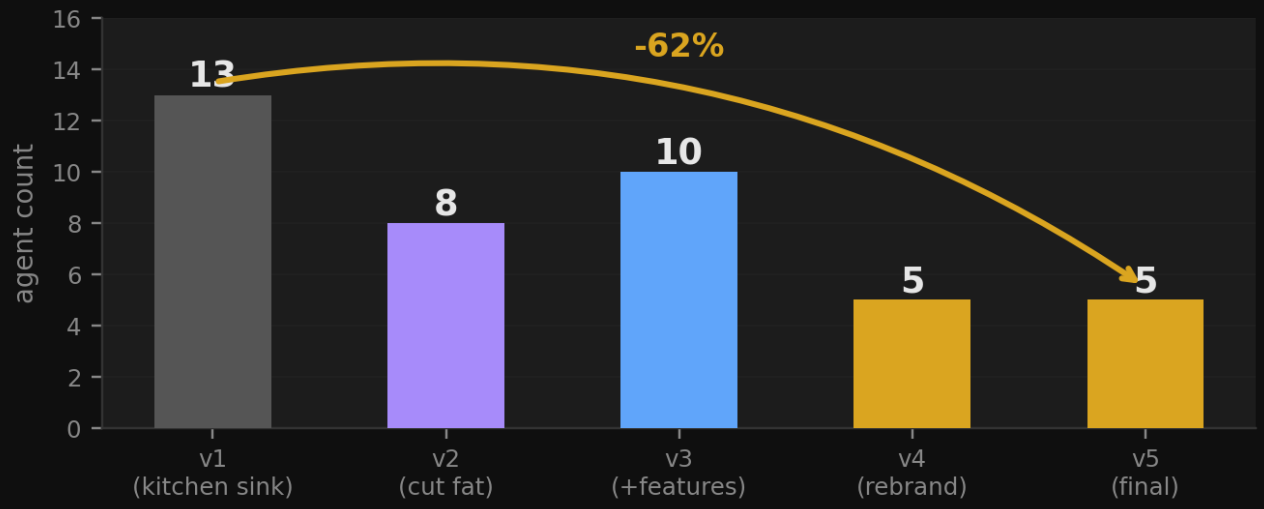
Gap tracking: problems don't disappear

Every eval surfaces ceiling gaps. Those gaps persist in the system until explicitly addressed. Escape velocity has been flagged since eval 1 and is still partially open. Identity was resolved after a dedicated sprint. New problems (visual personality) emerged as old ones closed. The mechanism is simple — append-only tracking — but it prevents the system from forgetting what's broken.



The system deleted itself into shape

Started with 13 agents doing overlapping work. Cut to 8. Added features, ballooned to 10. Rebrand forced a hard rethink — landed at 5 agents + 2 programs. Each surviving agent has clear ownership. The deleted agents weren't failures — they were learning. The final architecture is simpler because the early versions taught us what was actually needed.



Final architecture:

scout — landscape intelligence, market positions
strategist — portfolio calls, sprint planning
builder — the workhorse (gate/plan/build/experiment)
design-engineer — visual taste, UI/UX audits
sweep — daily triage, system health
meta — grades agents, fixes prompts, the training loop

Growth comes from the scoring, not the agents.

Agents are interchangeable. Prompts are tunable. The scoring rubric — what "good" means for your specific product and your specific users — is the irreplaceable piece. That's where domain expertise lives. That's what makes the experiment loop produce signal instead of noise.

github.com/rhinehart514/rhino-os

Open source. MIT license. Works with Claude Code natively.

All data from 2 projects over 2 days. N=65 experiments. Self-assessed scores. Interpret accordingly.